

给第一次读强化学习工程代码的你

GOPS 小白入门导读

看懂文件夹、PPO 与 SAC 的数据流，再拆开一个三自由度车辆轨迹跟踪环境

这份文档帮你解决三件事 ① 文件太多时从哪里下手；② 一条经验怎样从环境走到算法更新；③ 一周内需要读懂哪些文件。

建议阅读顺序 先看第 1-3 节建立地图，再按第 4-6 节追 PPO、SAC 与车辆环境，最后照第 7 节完成第一天练习。

范围 以本地 E:\GOPS 源码为准。仓库中约 346 个 Python 文件；本文按职责解释文件群，并详细指向本周真正要读的关键文件。无需一次读完所有实现。

入口脚本	创建组件	采样与缓存	算法更新	环境与评估
------	------	-------	------	-------

本周目标：能拿着源码，用自己的话讲清 PPO、SAC 各自的数据流，以及车辆跟踪环境怎样产生观测与奖励。

01 先有一张总地图

把 GOPS 想成一个实验流水线。环境模拟任务，策略网络给动作，采样器负责交互，训练器安排何时采样和更新，算法计算损失并调网络参数。训练脚本只负责把这些零件装起来。

obs 观测	策略 → act 动作	环境 → rew 奖励	采样器整理数据	算法更新参数
--------	-------------	-------------	---------	--------

名词	白话解释	在源码中常见的名字
环境 Environment	像一台可反复试验的模拟器；接收动作，返回结果。	env、create_env、reset、step
观测 Observation	策略此刻能看到的信息，不一定等于完整物理状态。	obs、observation_space
动作 Action	策略给环境的控制命令。	act、action、action_space
奖励 Reward	环境对本次动作/状态的评分。	rew、reward
策略 Policy	从观测得到动作的网络。	policy、actor
价值/评论网络	估计状态或状态-动作好坏；PPO 用 V，SAC 用 Q。	value、q1、q2、critic
训练器 Trainer	组织采样、更新、记录、评估和保存。	trainer、train、step
采样器 Sampler	真正调用环境 step 并收集经验。	sampler、sample、_step

最重要的一句话 训练脚本不直接实现 PPO 或 SAC 的数学细节；它选择组件。要看数据怎样流动，先从脚本底部的 create_* 和 trainer.train() 往下追。

文件名也能提供线索

例如 ppo_mlp_pendulum_onserial.py 可拆成：PPO 算法 / MLP 神经网络 / pendulum 倒立摆环境 / on-policy 串行训练。sac_mlp_pendulum_offserial.py 则是 SAC / MLP / 同一个倒立摆 / off-policy 串行训练。这里的 on/off 说的是样本的使用方式，不是环境开关。

02 顶层文件和文件夹分别做什么

位置	它负责什么	本周怎么对待
README.md	项目简介、安装和官方快速示例。	先看目标与 Quick Start。
setup.py / gops_environment.win.yml	包安装信息 / Windows 依赖环境配置。	知道用途即可，不从依赖表开始学算法。
本机配置说明.md / Open-GOPS.cmd	这台电脑的已验证环境 / 打开已激活终端。	需要运行时再查；当前解释器为 E:\miniconda\envs\gops\python.exe。
example_train/	训练入口：默认参数、创建组件、调用 train()。	本周只精读 PPO 和 SAC 两个 pendulum 文件。
gops/	框架源码主体。	按下表顺调用链读。
example_run/	加载已训练模型做闭环运行或绘图。	本周只知道用途；部分脚本需改结果路径和迭代号。
results/	训练输出：config.json、网络权重、日志、图表。	把它当实验记录，不当源码。
tests/	环境与模型的一致性检查。	扩展环境时再深入。
gops.egg-info/、__pycache__/、.git/	安装元数据、运行缓存、版本历史。	无需阅读。

gops/ 内部的 8 个功能区

子目录	角色	关键入口
create_pkg/	把字符串参数映射成真实对象的“装配区”。	create_env.py、create_alg.py、create_trainer.py
env/	任务环境、环境模型、包装器和向量环境。	本周看 env_gen_ocp/veh3dof_tracking.py
algorithm/	PPO、SAC、FHADP 等算法实现。	ppo.py、sac.py、base.py
apprfunc/	MLP 等策略/价值网络结构。	mlp.py；遇到 create_apprfunc 再打开
trainer/	训练循环、采样器、回放缓存与评估器。	on_serial_trainer.py、off_serial_trainer.py
utils/	初始化、日志、绘图、通用辅助函数。	init_args.py、plot_evaluation.py
sys_simulator/	读取策略，做闭环仿真/可选最优控制比较。	sys_run.py
optim/	优化器相关包位置。	当前主要是包初始化；算法基类也涉及优化器。

03 关键文件速查：按“工作岗位”找文件

A. 训练入口与组件装配

文件	第一次看它时只找什么
example_train/ppo/ppo_mlpPendulum_onserial.py	env_id=gym_pendulum 表示任务；algorithm=PPO 表示算法；trainer=on_serial_trainer。看底部 create_* → train()。
example_train/sac/sac_mlpPendulum_offserial.py	同一环境 gym_pendulum，algorithm=SAC，trainer=off_serial_trainer；对照 PPO 的不同参数。
create_pkg/create_env.py	create_env(env_id) 如何找到环境，并按参数加 wrapper。
create_pkg/create_alg.py	create_alg(algorithm) 如何找到 PPO 或 SAC 类。
create_pkg/create_sampler.py	sampler_name 如何选 OnSampler 或 OffSampler。
create_pkg/create_buffer.py	buffer_name 如何选 ReplayBuffer。
create_pkg/create_evaluator.py	如何创建评估器；第一遍不追 Ray 细节。
create_pkg/create_trainer.py	trainer 名称如何选具体训练器；on 分支不把 buffer 传入训练器。
create_pkg/create_apprfunc.py	policy/value 的网络类型如何变成 MLP 等实例。
create_pkg/create_env_model.py	需要可微环境模型的算法怎样创建模型；本周 PPO/SAC 主线暂不展开。
create_pkg/create_optim.py	优化器的创建入口；本周只需知道网络参数最终交给优化器更新。

B. 训练中的实际执行者

文件	一句话职责
trainer/on_serial_trainer.py	PPO 路线：采一批当前数据，直接交给算法更新。
trainer/off_serial_trainer.py	SAC 路线：预热缓存，持续采样、入库、随机回放和更新。
trainer/sampler/base.py	共同的 _step()：策略抽动作，调用 env.step()，整理一条 Experience。
trainer/sampler/on_sampler.py	PPO 数据：整理整批轨迹，计算 adv 与 ret。
trainer/sampler/off_sampler.py	SAC 数据：反复采集 Experience，送给缓存。
trainer/buffer/replay_buffer.py	保存 obs/act/rew/obs2/done 等，再随机抽出训练 batch。
trainer/evaluator.py	训练期间单独评估策略表现。

一个容易误解的细节 PPO 示例脚本里也调用 create_buffer()，但 create_trainer() 的 on-policy 分支不会把 buffer 交给 OnSerialTrainer；PPO 训练流程看当前采样批量即可。

04 PP0：一批新样本怎样变成一次更新

入口：example_train/ppo/ppo_mlp_pendulum_onserial.py。算法实现：gops/algorithm/ppo.py。先沿函数追数据，再看 PP0 论文中的截断目标函数。

当前策略采样	环境给出奖励	OnSampler 算 adv/ret	PP0 切小批量	计算损失并更新
--------	--------	------------------------	----------	---------

顺序	打开的文件 / 函数	你要找的东西
1	入口脚本末尾：create_*、trainer.train()	环境、算法、采样器和训练器怎样被组装。
2	on_serial_trainer.py: step()	sampler.sample_with_replay_format(); 随后 alg.local_update()。
3	sampler/base.py: _step()	policy(obs) → 分布采样动作与 logp → env.step(action)。
4	sampler/on_sampler.py: _sample()、_finish_trajs()	轨迹数据怎样整理；adv（优势）和 ret（回报目标）怎样由奖励和价值估计产生。
5	algorithm/ppo.py: local_update()、_compute_loss()	按小批量反复训练；新旧 logp 的比值进入截断策略损失；还有价值损失。

你现在只需会解释的三个量

- logp：采样器在采取动作时记录的对数概率；PP0 用它与更新时重新计算的概率作比较。
- adv：这次动作相对当前价值估计有多好；OnSampler._finish_trajs() 计算。
- ret：给价值网络学习的目标；同样在轨迹收尾时计算。

论文安排 第 2 次读 PP0 源码时，先用约 20 分钟看论文摘要、算法流程和截断目标函数。第 5 次比较两条数据流时再回看公式；第一遍不需要逐式推导。

05 SAC：经验怎样进入缓存、再被抽出来更新

入口：example_train/sac/sac_mlpPendulum_offserial.py。算法实现：gops/algorithm/sac.py。与 PPO 使用同一个 gymPendulum 环境，便于只比较数据处理路线。

采集 Experience	写入 ReplayBuffer	随机抽训练 batch	更新 Q 与策略	软更新目标 Q
---------------	-----------------	-------------	----------	---------

顺序	打开的文件 / 函数	你要找的东西
1	off_serial_trainer.py: __init__()	buffer_warm_size: 先采够若干条经验，才开始常规更新。
2	sampler/base.py: _step(); off_sampler.py: _sample()	环境返回一条 Experience，含当前/下个观测、动作、奖励等。
3	replay_buffer.py: add_batch()、sample_batch()	保存 obs/act/rew/obs2/done; 从已有经验随机抽 batch。
4	off_serial_trainer.py: step()	采样入库 → replay_batch_size 随机回放 → alg.local_update()。
5	algorithm/sac.py: local_update() 与损失函数	两个 Q 网络、策略网络、温度参数及目标 Q 网络怎样被使用。

三个“批量”别混淆

参数	白话意思
sample_batch_size	采样器一次向环境收集多少条新经验。
buffer_warm_size	开始常规训练前，缓存中至少先有多少条经验。
replay_batch_size	每次算法更新，从缓存里随机抽多少条旧/新经验。

别把 logp 用法混在一起：SAC 的 Experience / buffer 也包含采样时的 logp 字段，但这份实现的 Q 损失主要读取 obs、act、rew、obs2、done；策略更新会对当前策略重新采样并计算 new_logp。

06 PPO 与 SAC 并排看，其他算法先放在哪里

问题	PPO（本仓库示例）	SAC（本仓库示例）
训练器	OnSerialTrainer	OffSerialTrainer
采样器	OnSampler	OffSampler
更新前数据	当前策略刚收集的一批轨迹；采样器补出 adv、ret。	新经验先存 ReplayBuffer；训练器随机抽回放 batch。
网络重点	策略网络 + 状态价值网络。	策略网络 + 两个 Q 网络及目标 Q 网络。
第 1 遍只追	logp → adv/ret → PPO.local_update()。	obs/act/rew/obs2/done → buffer → SAC.local_update()。

仓库中还有很多算法，怎么理解这些文件夹？

example_train/ 的 ddpq、td3、dqn、dsac、trpo、fhdp、infadp、spil 等子目录，都是不同的“训练配方”；gops/algorithm/ 中同名文件才是对应的更新规则。先学会读一个配方如何调用框架，再逐步对比其他算法。

gops/apprfunc/ 的 mlp.py、cnn.py、poly.py、rnn.py 等负责网络形式；gops/env/env_gym/ 放 Gym 风格任务，env_ocp/ 与 env_gen_ocp/ 放控制任务的不同实现，后者更清楚地拆出 robot 和 context。env_matlab/ 与 Simulink 相关；wrapper/ 改造观测、动作或奖励接口；vector/ 支持批量环境。

学习顺序 本周先把 PPO、SAC 和一个车辆环境讲清。目录地图帮助你定位文件，不是要求你把所有算法和环境读一遍。

07 三自由度车辆轨迹跟踪：环境侧怎样工作

本周选择 `gops/env/env_gen_ocp/veh3dof_tracking.py`。它继承 `env_gen_ocp/pyth_base.py` 中的 `Env`；车辆动力学在 `robot/veh3dof.py`，参考轨迹在 `context/ref_traj.py`。

参考轨迹 + 车辆状态	生成 obs	策略给 steer / ax	Env.step() 推进	奖励 + 下一 obs
-------------	--------	----------------	---------------	-------------

要看的对象	位置	初学时理解到哪一步
车辆状态	<code>robot/veh3dof.py: Veh3DoF.step()</code>	<code>[x, y, phi, u, v, w]</code> 表示位置、朝向及运动状态；动作是 <code>[steer, ax]</code> 。
参考轨迹	<code>context/ref_traj.py: RefTrajContext</code>	生成未来参考点，提供期望位置、朝向与速度。
初始化	<code>veh3dof_tracking.py: reset()</code>	设置车辆初态与参考轨迹，返回初始 obs 和 info。
观测	<code>veh3dof_tracking.py: _get_obs()</code>	车辆相对参考轨迹的误差，加上未来参考点信息。默认 <code>pre_horizon=10</code> ，观测维数为 $6+4\times10=46$ 。
奖励/终止	<code>veh3dof_tracking.py: _get_reward()</code> 、 <code>_get_terminated()</code>	奖励惩罚位置/朝向/速度误差及动作；偏差过大时终止。
实际一步	<code>pyth_base.py: Env.step()</code>	先算本步奖励，再更新车辆和参考轨迹；终止时扣惩罚，最后返回下一 obs。

注意两个“状态” 车辆的完整物理状态在 `robot.state`；策略收到的是 `_get_obs()` 生成的观测。两者有关联，但不要当成同一个数组。

与 PPO/SAC 的关系：任何能接入相同环境接口的策略都可以概念上执行“obs → act → step”。本仓库没有针对这个新式 `veh3dof_tracking` 环境直接写好的 PPO/SAC 训练入口，因此本周只追环境侧，不把两个倒立摆示例的参数直接套到车辆上。

08 第一天照着做：60 分钟的逐步操作

准备：在 Cursor 打开 E:\GOPS。按 Ctrl+P 输入文件名快速打开；按 Ctrl+G 跳到行号。今天只读代码、画图，不必启动训练或先啃论文。

时间	操作	完成后应能回答
0-10 分	打开 PPO 入口；看默认 env_id、algorithm、trainer，以及文件底部 create_* 与 trainer.train()。	这个脚本准备训练哪个环境、哪种算法？
10-20 分	打开 SAC 入口的相同位置；把两者默认参数并排写下。	为什么是同一个 gym_pendulum，却选不同 trainer？
20-35 分	看 create_pkg/create_trainer.py 的 create_trainer()；只找 trainer 名称如何对应到 On/Off 类。	谁决定使用 OnSerialTrainer 还是 OffSerialTrainer？
35-50 分	看 on_serial_trainer.py 和 off_serial_trainer.py 各自 step() 中的采样行和 alg.local_update() 行。	谁调用采样器？谁调用算法更新？
50-60 分	画 6-8 个框的数据流图；在每框旁写对应文件名。	能否从入口讲到一次参数更新？

第一天可以直接抄下来的空白骨架

请自己填文件名 训练入口 → create_env/create_alg/create_sampler/create_trainer → trainer.train() → trainer.step() → sampler.sample() → alg.local_update()。再在旁边写 obs、act、rew、done 各是什么意思。

当天过关：不看本文，能从两个入口分别定位 train()、step() 和 local_update()；如果只能找到文件但说不出调用顺序，就再追一次箭头。

09 后四次怎么学，以及卡住时怎么办

次数	主任务	应留下的成果
第 2 次	PP0: OnSampler 如何产生 logp、adv、ret; PP0 如何更新。	PP0 单次迭代图; 论文只粗读摘要、流程和核心目标。
第 3 次	SAC: 采样、缓存、随机回放、Q 与策略更新。	SAC 单次迭代图。
第 4 次	veh3dof_tracking: 车辆状态、参考轨迹、观测、奖励。	变量表, 解释一次 step() 前后发生什么。
第 5 次	并排比较 PPO/SAC, 用车辆环境讲通用交互。	对照表 + 3-5 分钟口头讲解。

常见卡点，先用这些方法解开

- 文件太多：只追当前入口脚本实际调用到的函数。看到陌生目录先读表格的一句话职责。
- 看不懂参数：先问它属于环境、采样器、缓存、算法还是训练器；再看谁读取这个键。
- 论文看不进去：先画数据流，标出论文公式中的量在代码从哪里来；公式推导可以第二轮再补。
- 训练脚本跑不动：本周目标是读懂调用链，不靠长训练判断学习是否完成。既有三轮冒烟结果只能证明链路曾跑通。
- 车辆环境找不到 step()：任务文件继承了 pyth_base.py 的 Env.step()，本任务只重写观测、奖励、终止等函数。

最终验收：能回答这三个问题 ① env.step() 的输入、输出是什么？② PP0 的 adv/ret/logp 与 SAC 的回放经验从哪里来？③ 车辆跟踪的参考轨迹如何影响观测与奖励？

核对依据：本地 README.md、两份 pendulum 训练入口、gops/create_pkg、gops/trainer、gops/algorithm/pp0.py 与 sac.py、gops/env/env_gen_ocp/veh3dof_tracking.py 及其 robot/context/base 文件。文件位置以 E:\GOPS 为根。